

NourishNet

DMV Food Resource Connector

NAFSI 2026 Data Challenge | NourishNet Track

Team	Binxin Gao
Repository	github.com/binxingao666/nourish-net
Live App	binxingao666.github.io/nourish-net
Date	April 13, 2026

Table of Contents

1. Intended Users and Design Philosophy
2. User Interface: Features and Flow
3. Data Ingestion and Backend Architecture
4. Additional Data Sources
5. Scoring and Matching Engine
6. Prompt Engineering Experience
7. Future Improvements
8. Enhancements Beyond Kiro Output

1. Intended Users and Design Philosophy

NourishNet is built for three distinct stakeholder groups in the Maryland, Washington DC, and Virginia (DMV) metropolitan area. Each group faces its own set of information barriers when trying to engage with the food assistance ecosystem. NourishNet addresses these barriers through a single, unified application with role-specific interfaces.

1.1 Families Seeking Food Assistance

The primary user group consists of individuals and families experiencing food insecurity. These users need to quickly find nearby food pantries, distribution events, soup kitchens, and mobile markets. NourishNet serves them through a "Find Food" tab that allows ZIP code or geolocation search, filters by food type, dietary restrictions (halal, kosher, vegetarian, gluten-free), and walk-in availability. Results are ranked by a multi-factor scoring algorithm that prioritizes proximity, food relevance, and accessibility. Each result card shows operating hours, eligibility requirements, languages spoken on site, and provides one-click Google Maps directions.

Accessibility was a core design priority. The interface uses a clean visual hierarchy with large tap targets, emoji-enhanced labels, and plain language. Full translations are provided in English, Spanish, German, and Chinese to serve the DMV region's diverse population, where over 20% of households speak a language other than English at home.

1.2 Donors

Individuals and organizations wishing to donate food, funds, or supplies often lack clarity on which organizations are active near them, what they need, and how to contribute. NourishNet's "Give / Donate" tab addresses this by letting donors search by location, then filter by donation type (canned goods, fresh produce, prepared meals, monetary donations, clothing, hygiene products). Results highlight whether drop-off or pickup service is available, current organizational needs, donation hours, and tax-deductibility status.

1.3 Volunteers

The "Volunteer" tab connects people who want to donate their time with organizations that have open volunteer roles. Users filter by role type (food sorting, delivery driving, kitchen help, event coordination, intake assistance, warehouse work, gardening) and see each opportunity's schedule flexibility and requirements (age, background check, food handler certification). This helps volunteers find opportunities that genuinely match their availability and skills, rather than browsing multiple organizational websites.

2. User Interface: Features and Flow

2.1 Overall Layout

NourishNet uses a split-panel layout: a sidebar on the left containing search controls, filters, and a scrollable results list, and an interactive Leaflet map on the right showing all results as numbered markers. This layout was chosen because it lets users see both the ranked list and the geographic context simultaneously, which is critical for a resource-finding tool where distance matters.

2.2 Header and Navigation

The header features a frosted glass design with the application name and subtitle. A pill-shaped language switcher in the top right allows instant switching between English, Español, Deutsch, and . Below the header, three tab buttons switch between user modes. Each tab is color-coded (green for families, blue for donors, purple for volunteers) and the active tab color carries through the entire interface, including search buttons, rank badges, score indicators, and filter chips.

2.3 Search and Filtering

All three modes share a common search bar that accepts a ZIP code or provides browser-based geolocation via a location pin button. A distance slider (5 to 50 miles) is provided below the search bar. Role-specific filter panels appear below:

- **Family mode:** Food type chips (Groceries, Hot Meals, Fresh Produce, Canned Goods, Baby Food, Halal, Kosher), dietary need chips (Vegetarian, Halal, Kosher, Gluten-Free, Dairy-Free), and a walk-in-only checkbox.
- **Donor mode:** Donation type chips (Canned & Dry Goods, Fresh Produce, Prepared Meals, Money, Clothing, Hygiene Products), plus checkboxes for pickup service needed and tax-deductible only.
- **Volunteer mode:** Role type chips (Food Sorting, Delivery Driver, Kitchen Helper, Event Coordinator, Intake Assistant, Warehouse, Gardening).

2.4 Results and Result Cards

Search results appear as a scrollable list of cards ranked by match score. Each card displays the organization name, category badge, match percentage, distance, operating hours, and walk-in status. Clicking a card expands it to show full details: complete address, food types or donation types or volunteer roles, eligibility requirements, dietary options, languages spoken, and action buttons for directions, calling, website, and email. The corresponding map marker is also highlighted when a card is selected.

2.5 Interactive Map

The Leaflet map displays results as numbered circular markers (matching the rank numbers in the list) color-coded per user role. A home marker in orange shows the user's search location. Clicking a marker opens a popup with the organization name, address, hours, and a directions link. When a user selects a card in the results list, the corresponding marker scales up and gains a z-index boost to stand out visually. The map automatically re-centers when a new search is performed.

2.6 Responsive Design

On screens narrower than 768px, the layout switches to a vertical stack with the sidebar on top (limited to 55% viewport height) and the map below. Tab labels shrink to fit, and all touch targets are sized for mobile interaction. This ensures the tool is usable on the smartphones that many food-insecure households rely on as their primary internet device.

3. Data Ingestion and Backend Architecture

3.1 Hybrid Data Strategy

NourishNet employs a hybrid data approach combining curated static data with live API ingestion from multiple ArcGIS REST endpoints. This ensures the app never shows a blank screen (static data loads instantly) while also incorporating real-time information when available.

3.2 Static Data Layer

A curated dataset of 35+ food organizations across the DMV region is stored in `public/data/resources.json`. Each resource includes fields for all three user types: family-facing fields (food types, eligibility, dietary options, languages, walk-in status), donor-facing fields (accepted donation types, drop-off/pickup, tax deductibility, current needs), and volunteer-facing fields (roles, schedule, requirements, contact). A companion `zipcodes.json` file maps 300+ DMV ZIP codes to latitude/longitude coordinates for instant geocoding.

3.3 Live Data Ingestion

On application load, the data loader concurrently fetches from multiple DC Open Data ArcGIS REST API endpoints, including the Capital Area Food Bank Emergency Food Providers layer (691 records across DC/MD/VA), DC Summer Meal Sites (250 records), WIC Benefit Locations, Healthy Corner Stores, Group Meal Centers, and Anne Arundel County Food Pantries. Each endpoint has an independent 8-second timeout via `AbortController`, and failures are handled silently so that other sources can still contribute.

3.4 Data Merge and Deduplication

Static and live resources are merged into a single array, then deduplicated by a composite key of normalized organization name and address. Static data takes priority in deduplication because it contains richer metadata (volunteer roles, donation details) that the live APIs do not provide.

3.5 Data Schema

Each resource object conforms to the following schema:

Field Group	Fields	Purpose
Core	id, organizationName, address, lat, lng, city, state, zip, phone, email, website, operatingHours, category	Identity and location
Family	foodTypes, eligibilityRequirements, servicesOffered, dietaryOptions, languages, walkIn	Food seeker matching
Donor	acceptsDonations, donationTypes, donationDropOff, donationPickUp, donationHours, taxDeductible, donationNeeds	Donor matching

Volunteer	acceptsVolunteers, volunteerRoles, volunteerSchedule, volunteerRequirements, volunteerContact	Volunteer matching
Meta	sourceAttribution, lastUpdated	Provenance tracking

3.6 Geocoding Pipeline

ZIP code resolution follows a two-tier strategy. First, the app checks a local lookup table of 300+ DMV ZIP codes for instant results with zero network latency. If the ZIP is not in the table, it falls back to the Nominatim geocoding API (OpenStreetMap) with a NourishNet user-agent string. Browser geolocation is also supported as an alternative to ZIP entry.

4. Additional Data Sources

Beyond the CSV of food bank websites provided by the challenge organizers, NourishNet incorporates data from the following public sources:

Source	Usage
DC Open Data (ArcGIS REST API)	Live ingestion of CAFB emergency food providers, summer meal sites, WIC locations,
Capital Area Food Bank	Distribution center locations, operating hours, program types
Maryland Food Bank	Pantry network across Maryland
Anne Arundel County ArcGIS	67 food pantries + 2 soup kitchens with hours of operation
USDA SNAP Retailer Locator	SNAP-accepting food locations
PG County Food Equity Council	Prince George's County pantry listings
Montgomery County Food Council	MoCo food resource directory
UMD Extension Food Access	University food access programs
US Census / ACS	Demographic context for the DMV region
OpenStreetMap / Nominatim	Fallback geocoding service

All data sources are publicly accessible and documented in the repository's README.

5. Scoring and Matching Engine

NourishNet uses a multi-factor weighted scoring engine to rank resources for each user type. All scoring functions share a common proximity component based on haversine (great-circle) distance, with a 50-mile maximum radius and linear decay.

5.1 Family Matching

Factor	Weight	Description
Proximity	30%	Linear decay from 1.0 at 0 miles to 0.0 at 50 miles
Food Type Match	20%	Proportion of user-selected food types found in the resource
Dietary Match	15%	Proportion of dietary needs satisfied (halal, kosher, etc.)
Walk-in Score	10%	Binary: 1.0 if walk-in available and user requires it
Language Match	10%	1.0 if preferred language spoken on site, 0.5 otherwise
Services Breadth	15%	Normalized count of services offered (max at 5+)

5.2 Donor Matching

Factor	Weight	Description
Proximity	35%	Same haversine-based linear decay
Donation Type	25%	Proportion of user's selected donation types accepted
Pickup Convenience	15%	1.0 if pickup available and needed; 0.2 if not available
Tax Deductible	10%	Binary filter when user requires tax deductibility
Need Urgency	15%	1.0 if organization has stated current needs, 0.5 otherwise

5.3 Volunteer Matching

Factor	Weight	Description
Proximity	35%	Same haversine-based linear decay
Role Match	30%	Proportion of user's selected roles available at the organization
Requirements Fit	15%	Inverse of requirement count (fewer barriers = higher score)
Schedule Flexibility	20%	1.0 if "flexible" in schedule, 0.7 if specified, 0.5 if unknown

After scoring, results are filtered by the user's maximum distance setting, sorted by score (descending) with distance as a tiebreaker (ascending), and the top 15 are displayed.

6. Prompt Engineering Experience

The application was developed through a four-prompt sequence in Kiro, each building on the output of the previous step.

6.1 Prompt Sequence

Prompt	Purpose	Output
1. Project Init	Scaffold Vite + React + TS project with full directory structure	Working project skeleton with all config files and dependencies
2. Data & Engine	Build data files (35+ resources, 300+ ZIP codes), live API integration, and scoring engine	Complete data layer and three scoring functions
3. UI Components	Create all visual components: Header, tabs, three views, cards, map, and responsive CSS	Full working application with interactive features
4. Deploy & Docs	GitHub Actions workflow, README, PROMPTS.md, final build verification	Deployment-ready repository

6.2 Key Prompt Engineering Insights

- **Specificity in architecture prompts pays off.** Prompt 1 specified the exact directory structure, technology versions, and module boundaries. This prevented Kiro from making opinionated architectural choices that would require rework later.
- **Data prompts need schema-level detail.** Prompt 2 included the full TypeScript interface for each data object. Without this, Kiro tended to produce minimal schemas that only served one user type.
- **Visual design language should be described, not just requested.** Prompt 3 specified the exact color-coding scheme (green/blue/purple per role), component hierarchy, and interaction patterns. Abstract requests like "make it look professional" produced generic results.
- **Sequential prompts with clear handoff points are more reliable than monolithic prompts.** Each prompt assumed the previous output existed and specified only the new work.

6.3 Challenges Encountered

The main challenge was getting the Leaflet map integration right on the first pass. Kiro's initial output placed the map in a component that did not handle dynamic re-centering or marker updates correctly. The fix required specifying a `RecenterMap` helper component in the prompt that uses the `useMap()` hook to respond to center coordinate changes. Another challenge was ensuring the live DC GIS API call failed gracefully; the initial version did not include an `AbortController` timeout, causing the app to hang when the API was slow.

7. Future Improvements

While NourishNet already provides a functional and comprehensive tool, several enhancements would strengthen its real-world impact:

- **Real-time web scraping pipeline.** A scheduled scraping pipeline (using Puppeteer or Playwright) could ingest and parse the 24+ food bank websites from the challenge data sources spreadsheet, automatically extracting event dates, locations, hours, and eligibility from unstructured HTML.
- **Natural language search.** Rather than structured filters, users could type queries like "food near me that's open on Saturdays and has halal options" and a semantic parser would convert the intent into filter parameters.
- **Push notifications for events.** Families could subscribe to alerts when new distribution events are scheduled near their location.
- **Donor impact tracking.** Organizations could report donation fulfillment, allowing donors to see the impact of their contributions.
- **Volunteer hour logging and verification.** Integration with volunteer management systems to track hours and provide verification letters.
- **Additional languages.** The i18n framework supports adding new locales trivially. Priority additions would be Amharic, French, Korean, and Vietnamese, reflecting the DMV's major language communities.
- **Offline / PWA support.** Service worker caching would allow the app to function with limited connectivity.
- **Accessibility audit.** A formal WCAG 2.1 AA audit with screen reader testing would ensure the tool is truly inclusive.

8. Enhancements Beyond Kiro Output

The following improvements were made to the code after the initial Kiro output:

- **Expanded resource database.** Kiro's initial output included approximately 12 resources. The final dataset was expanded to 35+ organizations with comprehensive fields for all three user types, using data compiled from the challenge data sources spreadsheet and public organizational websites.
- **Live API integration hardening.** Added AbortController timeout (8 seconds), graceful fallback to static data, and a deduplication pipeline to merge live and static results without duplicates.
- **Multi-source live data.** Extended from a single DC GIS endpoint to 8 ArcGIS REST endpoints covering CAFB emergency food providers, summer meal sites, WIC locations, healthy corner stores, group meal centers, homeless services, and Anne Arundel County pantries.
- **Scoring engine refinement.** The initial scoring used only proximity and a single relevance dimension. The final engine uses 5 to 6 weighted factors per user type.
- **Chinese language support.** Added Simplified Chinese as a fourth locale alongside English, Spanish, and German.
- **iOS 26 liquid glass design.** Complete UI overhaul with frosted glass effects, Inter font, iOS system colors, pill-shaped controls, and smooth cubic-bezier animations.
- **Map marker interaction.** Added bidirectional selection sync between the result cards and map markers with scale animation and z-index management.